JAVA TUTORIAL – TEAM-BASED CODING ACTIVITY

THE MISSING EXAM PAPER: A DETECTIVE INVESTIGATION SYSTEM

1. Objective

In this tutorial, students will work in teams of four to develop a small Java-based Detective Investigation System. The activity is designed to provide hands-on practice with classes, objects, constructors, methods, arrays, control statements, and basic Git/GitHub collaboration.

Each team will develop one complete Java application by dividing the work into four separate Java files. Each student will be responsible for one part of the application and will contribute their work to a common GitHub repository using GitHub Codespaces.

---

2. Problem Statement

An important examination question paper has disappeared from the department office one day before the examination.

Five students were present in the department during the time of the incident. Your team has been assigned the task of developing a console-based Detective Investigation System in Java to investigate the suspects, examine clues, and identify the culprit.

The program should simulate an investigation using predefined data and choices.

The completed application should allow the detective to:

1. View the suspects.
2. Investigate a particular suspect.
3. Collect clues.
4. View collected clues.
5. Accuse a suspect.
6. Determine whether the accusation is correct.
7. Exit the investigation.

Note: User input using Scanner is NOT required for this activity.

---

3. Suspect Information

The following five suspects are involved in the investigation:

ID	Suspect	Location	Alibi
1	Alex	Computer Lab	Working on a project
2	Maya	Library	Studying
3	Rahul	Staff Room	Meeting a faculty member
4	Sara	Canteen	Having lunch
5	Arjun	Department Office	Collecting documents

The actual culprit should not be directly displayed to the detective at the beginning of the investigation.

---

4. Available Clues

The investigation contains the following clues:

1. The office door was opened at 2:15 PM.
2. CCTV shows someone entering the office.
3. A torn piece of paper was found near the printer.
4. A suspect's ID card was found inside the office.
5. The printer was used shortly before the question paper disappeared.

The clues should be stored using an array.

The program should also keep track of which clues have been collected.

---

5. Team Structure

Each team will consist of FOUR students.

Each student will be responsible for one Java file.

Student 1 – Suspect Management

File: Suspect.java

Responsible for creating and managing the Suspect class.

Student 2 – Clue Management

File: ClueManager.java

Responsible for creating and managing the clues.

Student 3 – Investigation and Accusation

File: Investigation.java

Responsible for investigating suspects and handling the accusation logic.

Student 4 – Main Program and Integration

File: DetectiveGame.java

Responsible for the main program, menu, program flow, and integration of all components.

---

6. Student 1 – Suspect Management

File: Suspect.java

The student should:

1. Create a class named Suspect.
2. Create suitable data members for:
    - Suspect ID
    - Suspect name
    - Location
    - Alibi
3. Create a constructor to initialize the data members.
4. Use the this keyword inside the constructor.
5. Create five Suspect objects.
6. Store the objects in an array.
7. Create a method to display the details of a suspect.
8. Create a method to display all suspects.

The class should be designed so that the other parts of the program can access the suspect information when required.

---

7. Student 2 – Clue Management

File: ClueManager.java

The student should:

1. Create an array containing the five clues.
2. Create a mechanism to track which clues have been collected.

3. Create a method to display the available clues.
4. Create a method to collect a clue using a predefined clue number.
5. Create a method to display all collected clues.
6. Ensure that a clue cannot be collected more than once.

---

8. Student 3 – Investigation and Accusation

File: Investigation.java

The student should:

1. Create a method to search for a suspect using the suspect ID.
2. Display the details of the selected suspect.
3. Create the accusation logic.
4. Compare the accused suspect with the actual culprit.
5. Allow a maximum of three accusation attempts.
6. Display an appropriate message when the accusation is correct.
7. Display an appropriate message when all three attempts are used.
8. Use suitable if-else, loops, break, and return statements where appropriate.

---

9. Student 4 – Main Program and Integration

File: DetectiveGame.java

The student should:

1. Create the main() method.
2. Create the objects required by the program.
3. Display the main investigation menu.
4. Use a switch statement for menu selection.
5. Use a loop to repeat the investigation.
6. Call the methods created by the other three students.
7. Integrate the suspect, clue, and investigation components.
8. Compile and test the complete application.
9. Work with the other team members to resolve integration errors.

---

10. Main Investigation Menu

The completed program should provide a menu similar to the following:

```
==================================
    DETECTIVE INVESTIGATION
==================================

1. View Suspects
2. Investigate Suspect
3. Collect Clue
4. View Collected Clues
5. Accuse Suspect
6. Exit
```

The program should repeatedly perform the selected operation until the investigation is completed or the detective chooses to exit.

---

11. Program Requirements

Option 1 – View Suspects

Display the details of all five suspects.

Option 2 – Investigate Suspect

Use a predefined suspect ID to identify and display the details of a particular suspect.

If the ID does not match any suspect, display an appropriate message.

Option 3 – Collect Clue

Allow the detective to select a clue using a predefined clue number.

The selected clue should be marked as collected.

A clue that has already been collected should not be collected again.

Option 4 – View Collected Clues

Display all clues that have been collected so far.

If no clues have been collected, display an appropriate message.

Option 5 – Accuse Suspect

Allow the detective to make an accusation using a predefined suspect ID.

The detective should have a maximum of three attempts.

If the correct suspect is identified:

CASE SOLVED!

You identified the culprit.

The missing question paper has been recovered.

If all three attempts are incorrect:

INVESTIGATION FAILED!

You have used all three attempts.

The culprit escaped.

Option 6 – Exit

Terminate the investigation.

---

12. Java Concepts to be Used

The completed project should demonstrate the following Java concepts:

- Classes
- Objects
- Reference variables
- Constructors
- this keyword
- Methods
- Arrays
- Array of objects
- if-else statements
- switch statement
- for loop

- for-each loop
- while or do-while loop
- break statement
- continue statement
- return statement
- Primitive data types
- Strings

Important

Scanner and user-input handling are **not required** for this activity because they have not yet been covered in class.

The program should therefore use predefined values to demonstrate the investigation.

---

13. Project Structure

Each team should have ONE GitHub repository containing the following files:

detective-investigation/

│

├── Suspect.java

├── ClueManager.java

├── Investigation.java

├── DetectiveGame.java

└── README.md

All four Java files must work together as one complete application.

---

14. GitHub Codespaces

Each team will work using GitHub Codespaces.

The repository should be opened in a Codespace where the team members can work on the project.

Each student should primarily work on their assigned file.

Student Responsibilities

Student 1 → Suspect.java

Student 2 → ClueManager.java

Student 3 → Investigation.java

Student 4 → DetectiveGame.java

Students should communicate with their team members before making changes that affect another student's file.

---

15. Git Workflow

Students should follow the basic workflow:

Write Code

↓

Test Code

↓

Commit Changes

↓

Push to GitHub

↓

Integrate with Team

↓

Test Complete Program

Useful Git commands:

git status

git add .

git commit -m "Add suspect management"

git push

Each student must make at least **one meaningful commit**.

---

16. Team Rules

1. Each student is responsible for their assigned Java file.
2. Every student must contribute code to the project.

3. Every student must make at least one meaningful Git commit.
4. Do not overwrite or delete another student's work without discussing it with the team.
5. Do not change agreed class names, method names, or parameters without informing the team.
6. Students should communicate with one another while integrating the files.
7. All four students must understand and be able to explain their own contribution.
8. The final program must compile successfully as one Java application.
9. The team must test the complete program before submission.

---

17. Suggested Team Workflow

Step 1 – Understand the Problem

All four team members should read and understand the complete problem statement.

Step 2 – Divide the Work

Each student should work on their assigned Java file.

Step 3 – Develop the Individual Components

Each student should write and test their own class and methods.

Step 4 – Commit the Work

Each student should commit their completed work to GitHub.

Step 5 – Integrate

The team should bring all four components together.

Step 6 – Compile and Test

The team should compile the complete application and identify any errors caused by integration.

Step 7 – Demonstrate

The team should demonstrate the complete Detective Investigation System.

Each student should explain their own contribution.

---

18. Final Deliverable

Each team must submit:

1. One GitHub repository.

2. Suspect.java
3. ClueManager.java
4. Investigation.java
5. DetectiveGame.java
6. README.md

The four Java files must work together as one complete Detective Investigation System.

The program should compile and execute successfully.

---

19. Team Demonstration

During the demonstration, each student should be able to explain:

- The part of the program they developed.
- The class/methods they created.
- The Java concepts used.
- How their file interacts with the other files.
- The Git commit they made.
- How the complete application works.

---

20. Learning Outcome

By completing this activity, students will be able to:

- Apply Java programming concepts to a practical problem.
- Design classes and objects.
- Use arrays and methods to organize a program.
- Apply control statements to implement program logic.
- Divide a larger problem into smaller modules.
- Work collaboratively on a common programming project.
- Use GitHub and Codespaces for basic collaborative development.
- Integrate independently developed Java files into a single application.
- Test and debug a complete Java program.